

Project Report — Nilai

Project Report — Nilai (AI-Assisted Academic Grading)

1. What this project is
2. How the project was created
3. How the project works — architecture & data flows
4. Business workflows & processes
5. Prompt / conversation summary
6. Default credentials

Project Report — Nilai (AI-Assisted Academic Grading)

Report generated: October 2026. Repository: `umbima-smt1-tugas-generativeai`.

1. What this project is

Nilai is a web platform for running academic courses and assessments end-to-end: courses and assignments, student submissions, rubric-based grading, and optional **AI-assisted grading** (LLM + RAG over PDF/DOCX chunks stored in pgvector). Three roles: Admin, Dosen (lecturer), and Mahasiswa (student). The display name, footer, and logo are admin-configurable (default: “MiniCourse”). See `README.md` for the full feature list.

Stack: Next.js 16 (App Router, server actions), React 19, Tailwind CSS 4, PostgreSQL 17 + pgvector, ExcelJS, Docker Compose.

2. How the project was created

The project was built **iteratively by prompting an AI coding agent** (opencode, powered by a large language model) from the command line, with a human steering, reviewing, and committing each stage. It was not generated in a single prompt — it grew through a repeated loop:

1. **Prompt** — the human described a feature or change in natural language.
2. **Implement** — the agent explored the codebase, edited files, and kept to the existing design system (Moodle Boost-like UI, Indonesian UI copy).
3. **Static checks** — `tsc --noEmit` and `eslint src --max-warnings=0` after every change.
4. **Automated verification** — a protocol run against a real Docker environment: an HTTP action-suite (~64 checks with SQL assertions), page spot-checks across all roles (~30 pages), login/session tests (9 checks), and an end-to-end Playwright browser walkthrough (38 checks, including the branding and Excel-import flows).
5. **Restore** — the database was reset to its pristine seeded state (6 users / 2 periods / 3 classes / 8 enrollments / 4 submissions / 2 grades) after every full test run.
6. **Commit** — only on explicit human request; each feature stage became one commit.

Development stages (from git history)

Stage	What was built	Commit
1	Core app: roles & auth, courses, topics, 4 assignment types (PG/essay/PDF/DOCX), submissions, rubric grading, AI grading pipeline (extraction → embedding → RAG → LLM), admin/lecturer/student areas, seed data — plus academic structure: Periode , Kelas (section) , and student placement	8ddebcc
2	Admin-configurable branding (“Aplikasi” tab) and Excel user import (preview → confirm, skip existing emails, template download)	67fb991
3	<code>README.md</code> with features, quickstart, configuration, and deployment guide	8df6d2c
4	Project report (this document) as PDF	ab168c1

3. How the project works — architecture & data flows

Figure 1 shows the system architecture: the browser talks to a single Next.js app; all permissions are enforced server-side (route proxy + `requireRole` in every server action); data lives in one PostgreSQL database (with pgvector for embeddings); AI calls go to external OpenAI-compatible APIs configured in `/admin/settings`.

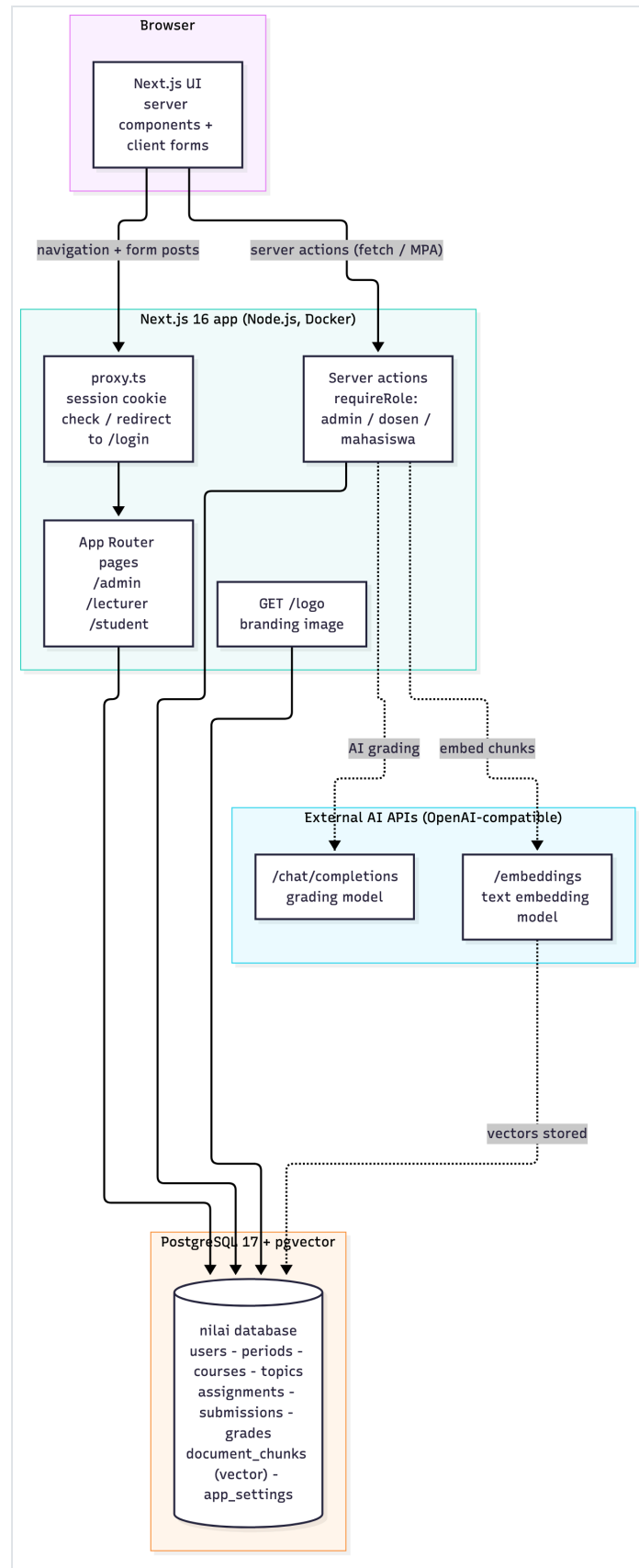


Figure 1: System architecture — browser, Next.js app, database, external AI APIs

Figure 2 shows the AI grading pipeline: a submitted document is text-extracted (never stored raw), chunked and embedded into pgvector; at grading time the top-k chunks plus the dosen's rubric are sent to the LLM, which returns nilai, predikat, and feedback recorded as a draft grade (`source = model`) until published.

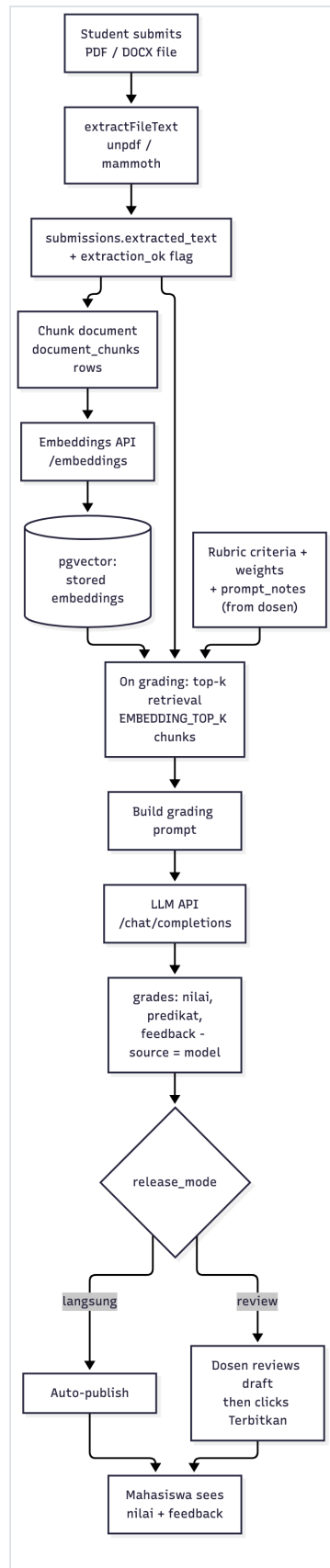


Figure 2: AI grading pipeline — extraction, embedding, retrieval, LLM grading, publish

4. Business workflows & processes

Course setup (Admin → Dosen). Admin creates the academic period, creates the class (section), assigns the dosen, and places students. The dosen then creates a pertemuan (topic), authors the assignment (type

PG/Essay/PDF/DOCX), adds questions with an answer key or a weighted rubric (weights sum to 100), and chooses a due date and release mode. The assignment is then open for attempts.

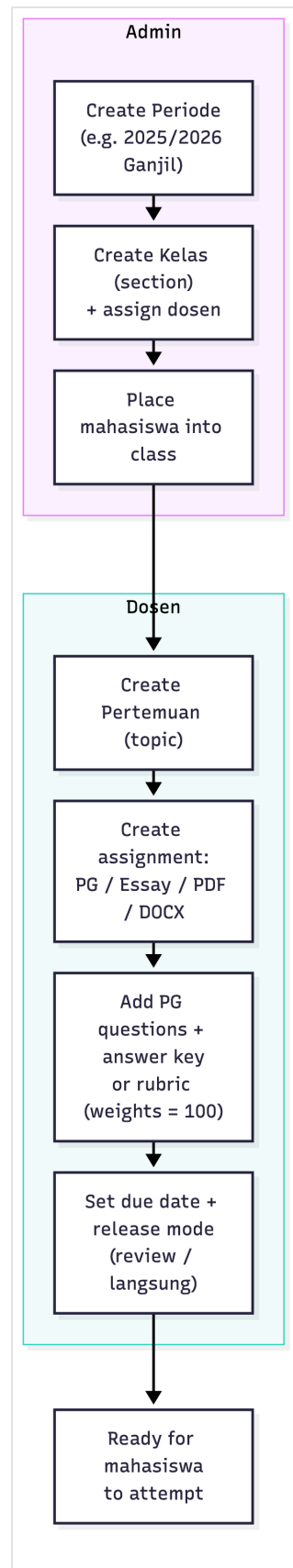


Figure 3: Business process part 1 — course and assignment setup

Attempt, grading, and release (Mahasiswa → grading → result). A student attempts the assignment once (unique per student; the attempt locks after submit). The grade is produced by one of three sources: PG

answers are code-graded against the answer key, PDF/DOCX submissions go through the AI RAG pipeline (Figure 2), and essay/manual grading scores each rubric criterion with feedback. The grade exists first as a **draft**; depending on the release mode, *langsung* publishes it automatically while *review* requires the dosen to click Terbitkan. The student then sees nilai, predikat (A-D), and feedback; any later change is snapshotted in grade revisions.

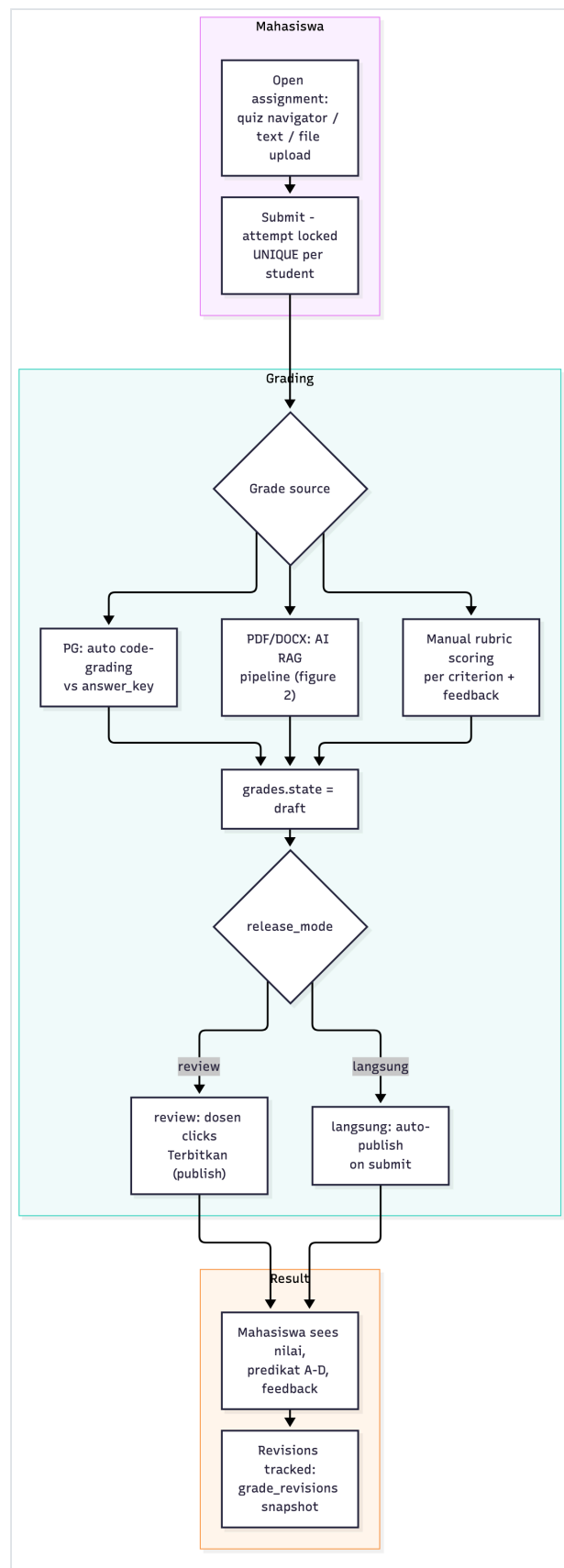


Figure 4: Business process part 2 — attempt, grading sources, publish, result

User onboarding (Admin). New users are added manually or in bulk via Excel: the admin uploads an `.xlsx`, the server parses and validates it (role/status aliases, email format), shows a per-row preview (*siap diimpor / sudah terdaftar / gagal*), and on confirmation inserts only valid new rows — existing emails are skipped, never overwritten. Imported accounts start with the default password and are then placed into a class via placement.

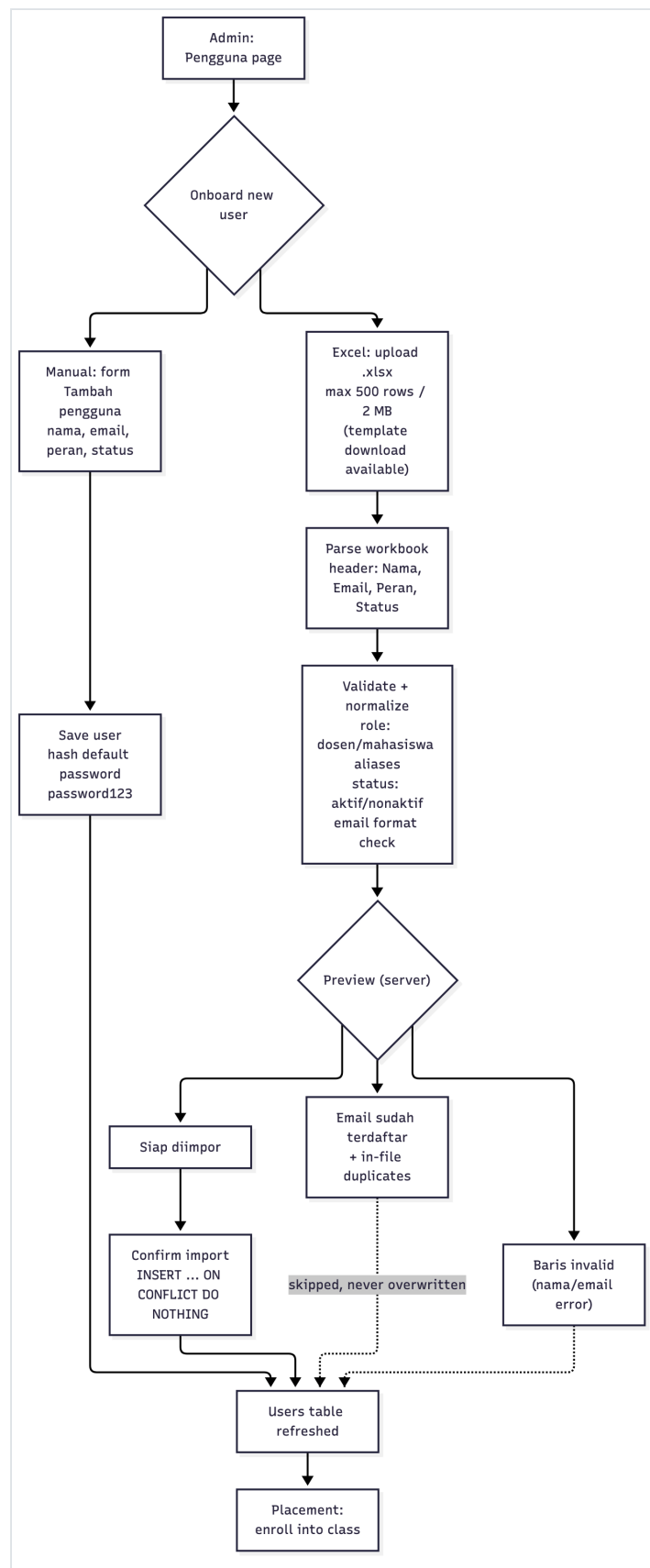


Figure 5: Business process part 3 — user onboarding and Excel import

5. Prompt / conversation summary

Paraphrased summaries of the prompts that drove the project (wording condensed; early-session wording reconstructed from the session record). Credentials/API keys are omitted.

1. **Initial build** — “Build an academic grading app: admin, dosen, mahasiswa roles; courses with topics; assignments (PG, essay, PDF, DOCX) with submissions, rubrics, and grades; AI-assisted grading using an LLM + embeddings/RAG; Moodle Boost-style UI with Indonesian copy; seed data for demo.” (*Early stages also fixed strict process rules: answer in English, UI text in Indonesian, no emoji, never commit unless asked, always verify before finishing.*)
2. **Academic structure** — “Add admin management for academic periods (Periode) and classes with sections (Kelas), let admins place students into classes, show period/section read-only to dosen and on all course cards.”
3. **Rebrand** — “Change the brand to MiniCourse.”
4. **Configurable branding** — “Let the admin configure the app name, footer text, and logo from a new ‘Aplikasi’ admin tab.”
5. **Excel import** — “Import users from Excel: admin uploads an .xlsx, sees a preview first, then confirms; skip emails that already exist instead of overwriting; provide a template download.”
6. **Verification & delivery** — “Continue if you have next steps...”, “commit, and where we can deploy this?”, “regenerate readme.md with project summary + how to deploy”, “commit”, requests for this report and its diagrams.

Process constraints given along the way (kept throughout): UI strings in Indonesian, English assistant replies, commit only on request, full test protocol green before any stage was called done.

6. Default credentials

Demo accounts (seeded on first boot, development/demo use — change or disable in production):

Role	Email	Password
Admin	audyah@nilai.test	password123
Dosen	ramadan@nilai.test	password123
Mahasiswa	aldy@nilai.test	password123
Mahasiswa	miratil@nilai.test	password123
Mahasiswa	ismail@nilai.test	password123
Mahasiswa	fitri@nilai.test	password123

Database (Docker Compose defaults): user `nilai`, password `nilai`, database `nilai`, host port `5432`. Application port: `3000` (`3001` in dev mode).

Secrets not included in this report: the session-cookie secret and the LLM/embedding API keys live in `.env.local` (see `.env.example` for variable names) and/or `/admin/settings`. They must be set by the operator — never reuse development values in production.